SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1108

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: Any 4 **Total Marks:40**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I Deepa lakshana.B (192471054) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

ANALYTICAL PROBLEM SOLVING

1. Use Case Diagram Problem

a) Actors

- Patient
- Doctor
- Administrator

b) Primary Use Cases

- Book Appointment
- View Appointment
- Update Medical Record
- Manage Hospital Resources
- Login

c) Relationships

- <<**include**>>: Book Appointment → Login
- <<**include**>>: Update Medical Record → Login
- <<**include**>>: Manage Hospital Resources → Login
- <<**extend**>>: View Appointment → Book Appointment
- **Generalization**: User → Patient, Doctor, Administrator

```
d)use case diagaram              User
                   ▲
   -------------------------------
   |          |          |
 Patient     Doctor    Administrator

   |          |          |
   |          |          |
 (Book Appointment) (Update Medical Record)
   |          |          |
   |<<include>>  |<<include>>   |<<include>>
   |          |          |
      (Login)
         |
    (View Appointment)
      <<extend>>
         |
 (Manage Hospital Resources)
```

Benefits

- Clearly identifies **actors** and **system functions**.
- Shows relationships using **include, extend, and generalization**.
- Improves **requirement analysis, communication, and system design**.

---

2. Class Diagram Problem

a) Classes with Attributes and Methods

Book

- Attributes: BookID, Title, Author
- Methods: issueBook(), returnBook()

Member

- Attributes: MemberID, Name
- Methods: borrowBook(), returnBook()

Librarian

- Attributes: LibrarianID, Name
- Methods: addBook(), removeBook()

IssueRecord

- Attributes: IssueDate, ReturnDate
- Methods: createRecord(), closeRecord()

---

b) Relationships

- **Association:** Member ↔ Book (borrows books)
- **Aggregation:** Librarian ◇— Book (manages books)
- **Inheritance:** Person → Member, Librarian

---

c) Multiplicity

- **Member (1) → (0..*) IssueRecord**
- **Book (1) → (0..*) IssueRecord**
- **Librarian (1) → (0..*) Book**

d)class diagram

```
        Person

          ▲

    -----------------

    |           |

  Member      Librarian

    |           ◇
```

```
   |            |
   |------------> Book
   |
   |
IssueRecord
   |
   |
  Book
```

Benefits

- Clearly defines **classes, attributes, and methods**.
- Shows **association, aggregation, and inheritance** relationships.
- Improves **system design, reusability, and maintainability**.

---

3. Sequence Diagram Problem

a) Objects Involved

- Customer
- Food Ordering System
- Payment Gateway
- Restaurant

b) Message Flow

1. Customer → Select Food
2. Customer → Place Order
3. System → Payment Gateway (Process Payment)
4. Payment Gateway → System (Payment Success)
5. System → Restaurant (Send Order)
6. Restaurant → System (Confirm Order)
7. System → Customer (Order Confirmation)

c) Activation & Lifelines

- Each object has a **lifeline** (vertical dashed line).
- **Activation bars** appear while processing requests such as payment and order confirmation

d) Sequence Diagram Customer   System   Payment Gateway   Restaurant

```
  |          |            |             |
  |--Select Food-------------> |             |
  |--Place Order-------------> |             |
  |          |--Process Payment----------->   |
```

```
 |          |<--Payment Success-----------   |
 |          |--Send Order------------------------------>|
 |          |<-------------Order Confirmed--------------|
 |<--Order Confirmation--------|              |
```

Benefits

- Shows the **order of interactions** between objects.
- Clearly represents **message flow, lifelines, and activations**.
- Improves **system understanding and communication**.

---

4. Activity Diagram Problem

a) Activities

- Submit Application
- Verify Documents
- Pay Admission Fee
- Confirm Enrollment

b) Decision Nodes

- Documents Verified?
  - **Yes** → Fee Payment
  - **No** → Reject Application
- Fee Paid?
  - **Yes** → Confirm Enrollment
  - **No** → Pending/Cancel Admission

c) Parallel Processing

After application submission:

- Document Verification
- Eligibility Check

  These two activities can run **in parallel** before proceeding to fee payment

d) Activity Diagram

```
Start
  |
Submit Application
  |
  +---------------------------+
  |                          |
Verify Documents      Check Eligibility
  |                          |
  +------------+--------------+
               |
      Documents Verified?
        /              \
      Yes               No
       |                |
  Pay Admission Fee   Reject Application
       |
    Fee Paid?
    /      \
  Yes       No
   |        |
```

Confirm Pending/
Enrollment Cancel
|
End

Benefits

- Shows the complete **workflow** clearly.
- Includes **decision nodes** for approval/rejection.
- Uses **parallel processing** to improve efficiency.
- Makes the admission process **faster and easier to understand**.

5. State Diagram Problem

a) States

- Initial State
- Idle
- Product Selected
- Payment Received
- Product Dispensed
- Transaction Completed
- Final State

b) Events Triggering Transitions

- **Start** → Idle
- **Select Product** → Product Selected
- **Make Payment** → Payment Received
- **Dispense Product** → Product Dispensed
- **Complete Transaction** → Transaction Completed
- **Finish** → Final State

c) Initial and Final States

- **Initial State:** Start
- **Final State:** End

d) State Transition Diagram● (Start)
|
Idle
|
Select Product
|
Product Selected
|
Make Payment
|
Payment Received
|
Dispense Product
|
Product Dispensed
|
Complete Transaction
|
Transaction Completed

|
◎(End) **Benefits**

- Clearly shows all **states** and **transitions**.
- Represents **system behavior** based on events.
- Improves **system design, testing, and maintenance**.

---

6. Component Diagram Problem

a) Major Components

- User Management
- Course Management
- Assessment Module
- Content Delivery Module
- Notification Service

b) Interfaces

- **User Management** → Authentication/Login
- **Course Management** → Course Access
- **Assessment Module** → Exam & Results
- **Content Delivery Module** → Learning Content
- **Notification Service** → Email/SMS Notifications

c) Dependencies

- User Management → Course Management
- Course Management → Content Delivery Module
- Course Management → Assessment Module
- Assessment Module → Notification Service

d) Component Diagram

```
           +----------------------+
           |  User Management     |
           +----------------------+
                   |
                   v
           +----------------------+
           | Course Management    |
           +----------------------+
            /                \
           v                  v
+----------------------+   +----------------------+
| Content Delivery     |   | Assessment Module    |
+----------------------+   +----------------------+
                             |
                             v
                  +----------------------+
                  | Notification Service |
```

Benefits

- Identifies major **system components**.
- Shows **interfaces and dependencies** between modules.
- Improves **modularity, scalability, and maintainability** of the system.

---

7. Deployment Diagram Problem

a) Hardware Nodes

- Roadside Sensor
- Cloud Server
- Traffic Authority Computer (Web Dashboard)

b) Software Components

- Roadside Sensor: Data Collection Software
- Cloud Server: Traffic Monitoring System, Database, Alert Service
- Traffic Authority Computer: Web Dashboard

c) Communication Links

- Roadside Sensor ⇄ Cloud Server (Internet/Wireless)
- Cloud Server ⇄ Traffic Authority Computer (Web Network**)**

d) Deployment Diagram

```
+--------------------+
| Roadside Sensor    |
|--------------------|
| Data Collection SW |
+--------------------+
        |
   Internet/Wireless
        |
        v
+---------------------------+
| Cloud Server              |
|---------------------------|
| Traffic Monitoring System |
| Database                  |
| Alert Service             |
+---------------------------+
        |
     Web Network
        |
        v
+-------------------------+
| Traffic Authority PC    |
|-------------------------|
| Web Dashboard           |
```

Benefits

- Shows **hardware nodes** and **software deployment**.
- Defines **communication links** between devices.
- Improves **system scalability, monitoring, and maintenance**.

RUBRICS FOR CALCULATION

ANALYTICAL PROBLEM SOLVING

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation